VCS-GIT
1-Controllo di versione

Prof. Marcello Missiroli

DI CHE SI PARLA?

Quando un progetto software diventa complesso, anche la sua gestione è complicata.

Il controllo revisioni permette di gestire tutte le modifiche apportate ai documenti del progetto, archiviando tutte le versioni del progetto sin dal suo inizio

È uno strumento essenziale nello sviluppo moderno, e non usarlo può solo portare a conseguenze spiacevoli...

No VCS?

AHI AHI AHI!

1.

Il problema

Bigger is not always better

Come si programma?

In the small

Un programmatore, un programma piccolo, limitato nel tempo e nelle dimensioni. Una stazione di lavoro.

Metodologia di sviluppo informale.

In the large

Un piccolo team, coordinato, co-locato.

Software decomposto in più moduli, su più versioni, su più configurazioni.

Metodologia di sviluppo strutturata

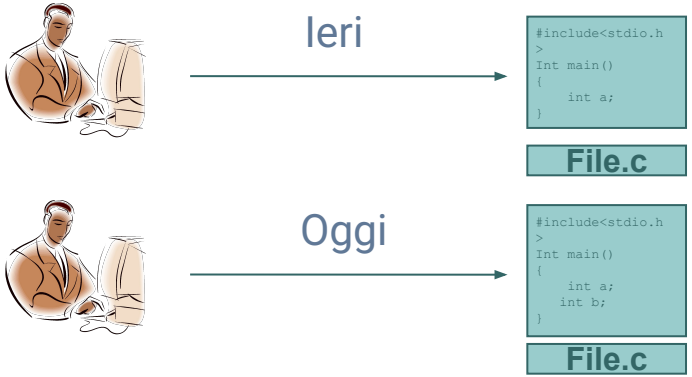
In the many

Grande team eterogeneo e distribuito. Grandi grandi sistemi sw.

Metodologia di sviluppo fortemente strutturata.

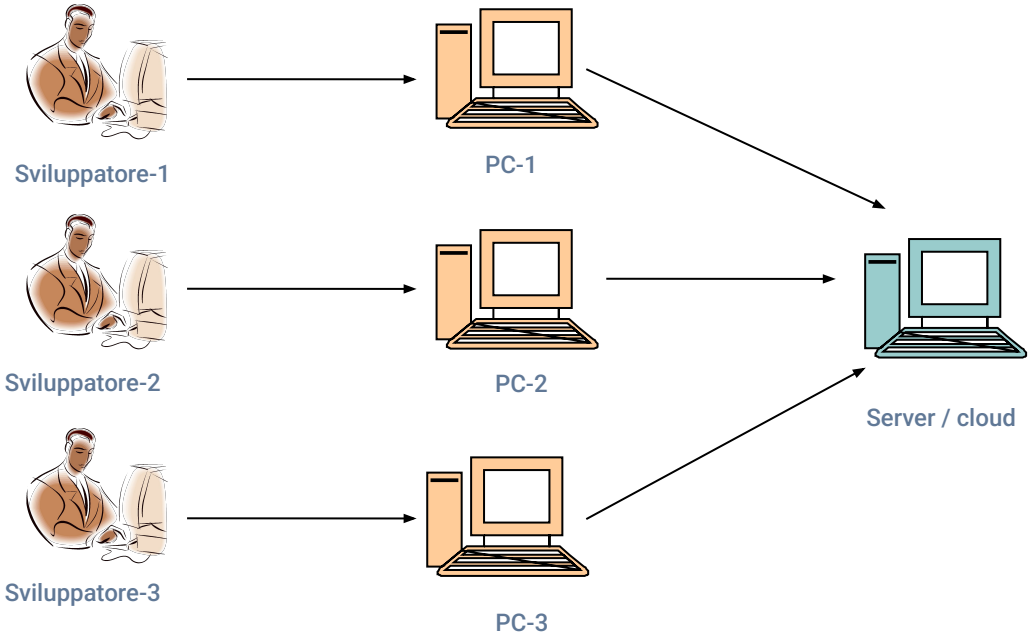
Programming in the small

- Difficile recuperare le versioni più vecchie basandosi sulla data.
- Difficile stabilire le differenze tra le versioni di uno stesso file.



Programming in the large/many

- Non c'è modo di recuperare le versioni precedenti
- Non c'è modo di vedere le differenze tra le versioni.
- Processo di fusione versioni manuale, lento e di dubbio successo.
- Sovrascritture accidentali



Version Control System

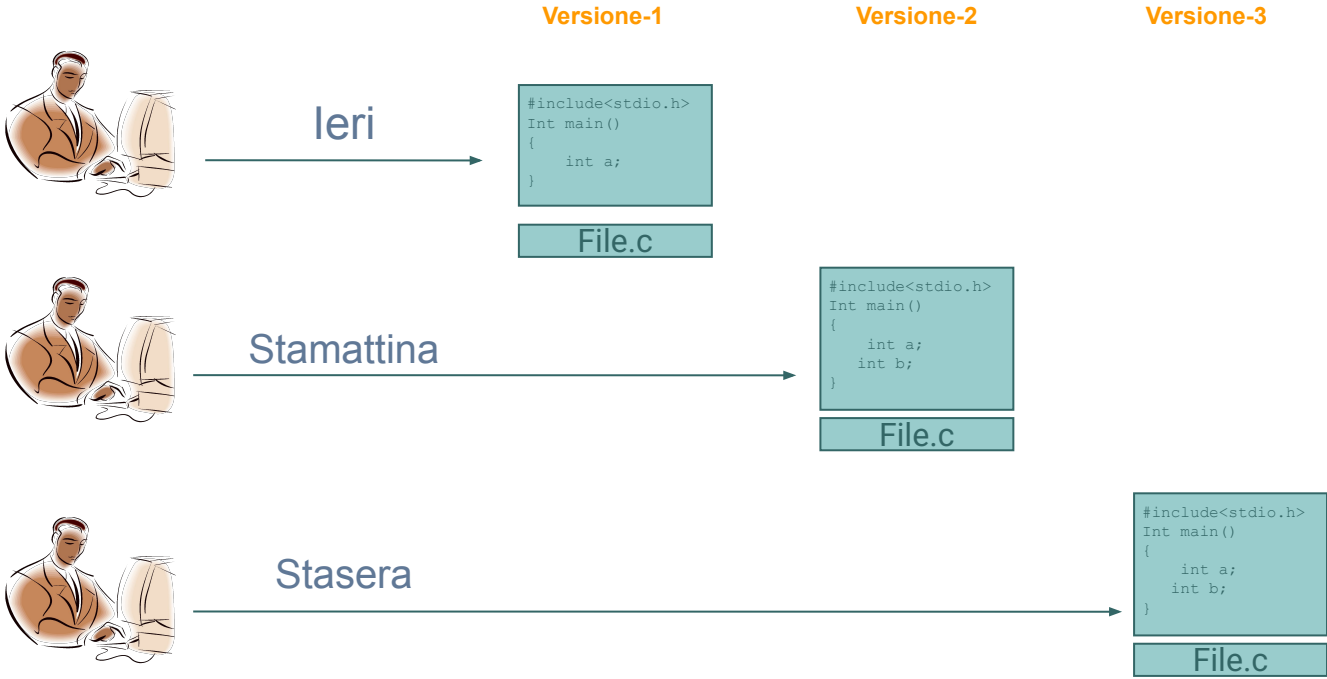
Detto anche Versioning, o Revision Control System o Source control management.

Revision control (also known as version control, source control or (source) code management (SCM)) is the management of changes to documents, programs, and other information stored as computer files.

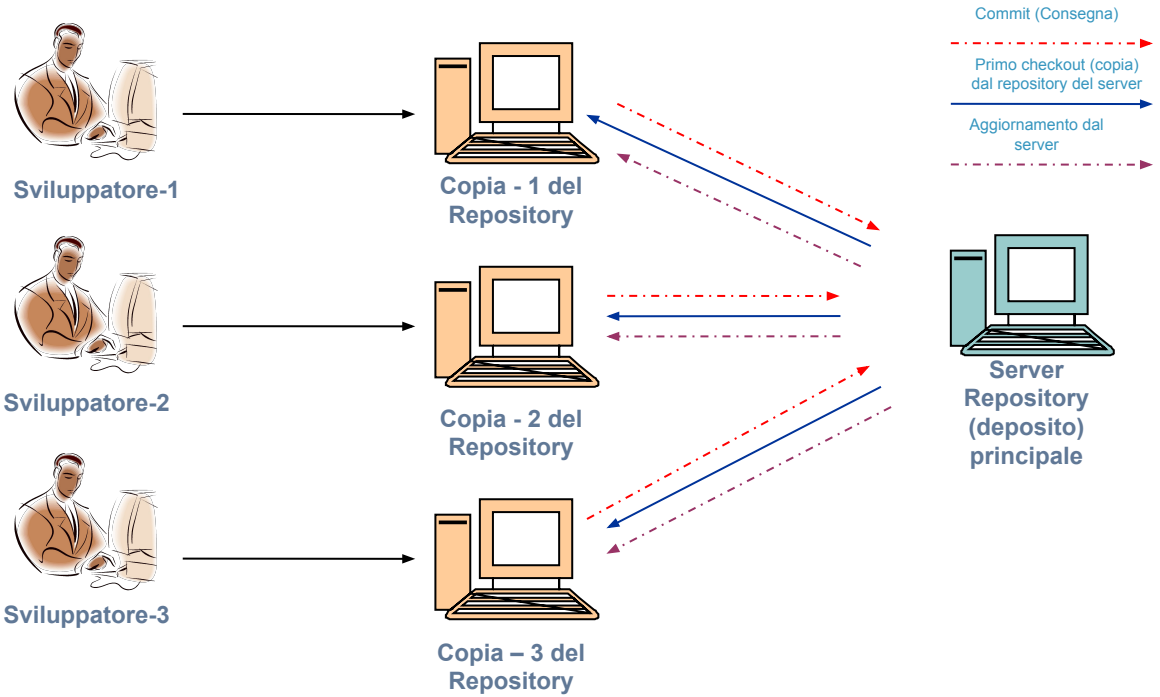
In pratica, VCS ci dà...

- Un posto dove memorizzare le varie versioni del codice (e tutti i documenti correlati), un backup multiplo e flessibile.
- Possibilità di “tornare indietro” in caso di errore e tracciare le modifiche
- Sviluppo parallelo facilitato, potendo lavorare su più versioni contemporaneamente

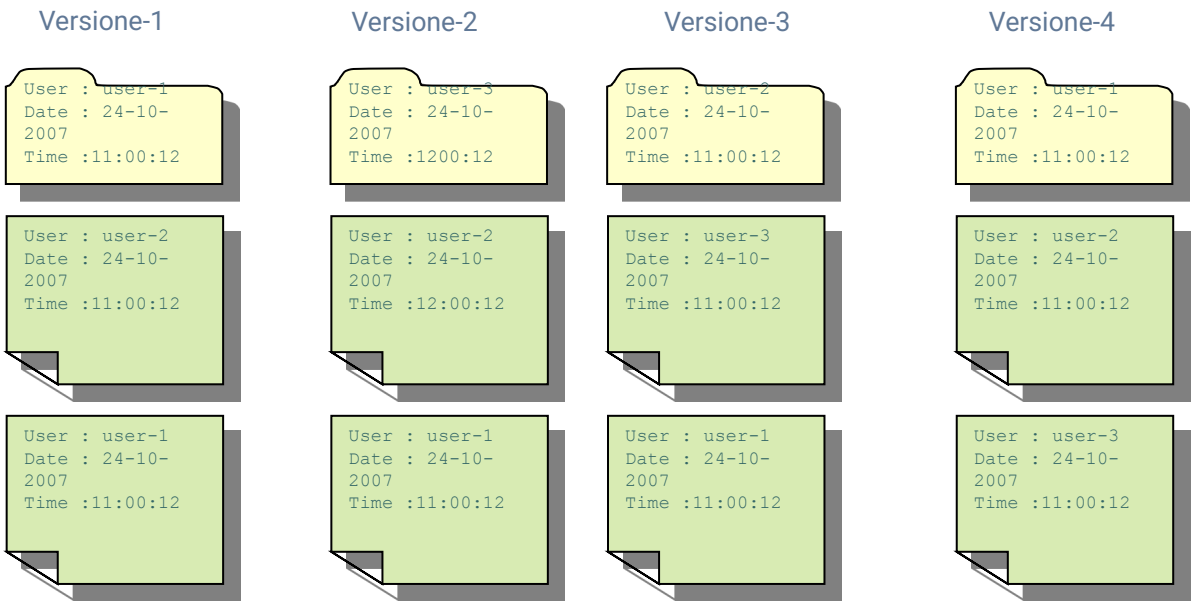
Programming in the small con RCS



Programming in large/many con RCS



Concetto base: il repository



Ma come funziona?

- Git è basato sul principio di UNIX diff.
- Invece di memorizzare *copie* di un file, memorizza le *differenze* tra un file e l'altro, indicando le righe inserite, modificate, cancellate.
- Vantaggio: **minima occupazione di spazio**, **rapidità di passaggio da una versione all'altra**.
- Svantaggio: funziona a dovere **solo con file di testo**.

In many ways, you can just see Git as a filesystem -- it's content-addressable, and it has a notion of versioning, but I really designed it coming at the problem from the viewpoint of a filesystem person (hey, kernels is what I do), and I actually have zero interest in creating a traditional SCM (source control management) system.

Plù tecnicamente

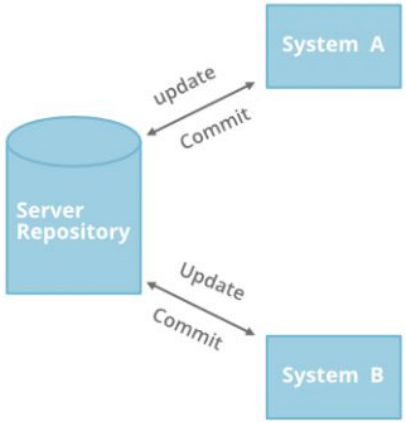
- Git possiede le seguenti strutture.
  - Un indice modificabile (un punto di connessione tra il database degli oggetti e la directory di lavoro)
  - Una database **immutabile, append-only**
- Per git esistono cinque tipi di oggetti: blob, tree, commit,tag, packfile, identificati da un hasj SHA-1.
- Tutto è contenuto nella cartella .git

2.

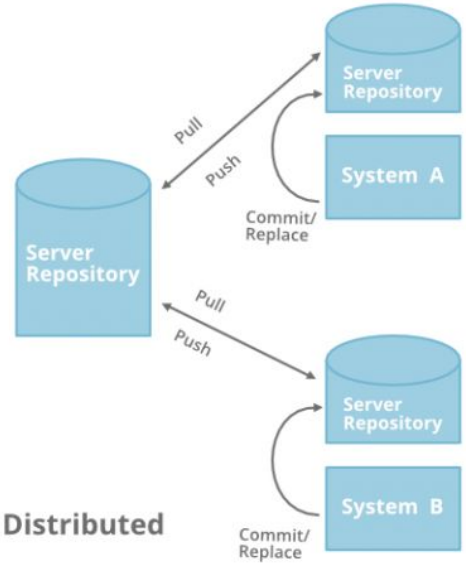
Git

“You stupid git!”

Tipologie di VCS



Centralized



Distributed

RCS

RCS is created by Walter Tichy

1982

BITKEEPER + SVN

BitKeeper is created by Bitmover Inc and SVN is created by Collabnet Inc

2000

1972

SCCS

SCSS is created by Marc Rochkind

1986

CVS

CVS is created by Dick Grune

2005

MERCURIAL + GIT

Mercurial is created by Matt Mackall and Git is created by Linus Torvalds

RCS timeline

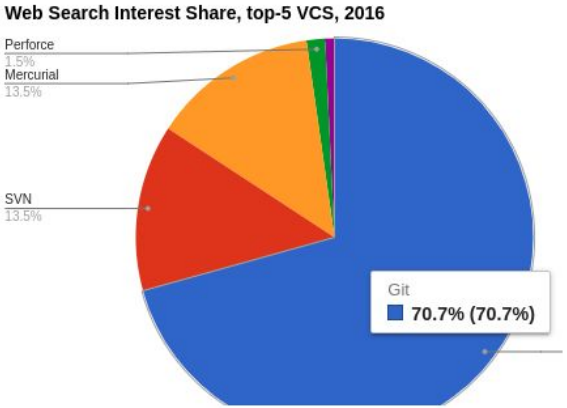
Confronto

SCCS & RCS (‘82)

CVS (‘86)

Subversion (2000)

GIT (2005)



Gitstoria

Git è figlio di LInux. Per molto tempo lo sviluppo del kernel Linux le modifiche al software venivano passate sotto forma file compressi. Nel 2002, si iniziò ad utilizzare un sistema DVCS proprietario chiamato BitKeeper.

Nel 2005 fu revocato l'uso gratuito di BitKeeper con grandi polemiche. Ciò indusse Linus Torvalds sviluppare uno strumento proprio. Dopo 15 giorni di sviluppo, Git fu pronto.

.

Git si è evoluto e maturato. È incredibilmente veloce, è molto efficiente con progetti grandi e ha un incredibile sistema di ramificazioni, per lo sviluppo non lineare. Oggi si stima lo usino almeno l'85% dei professionisti.

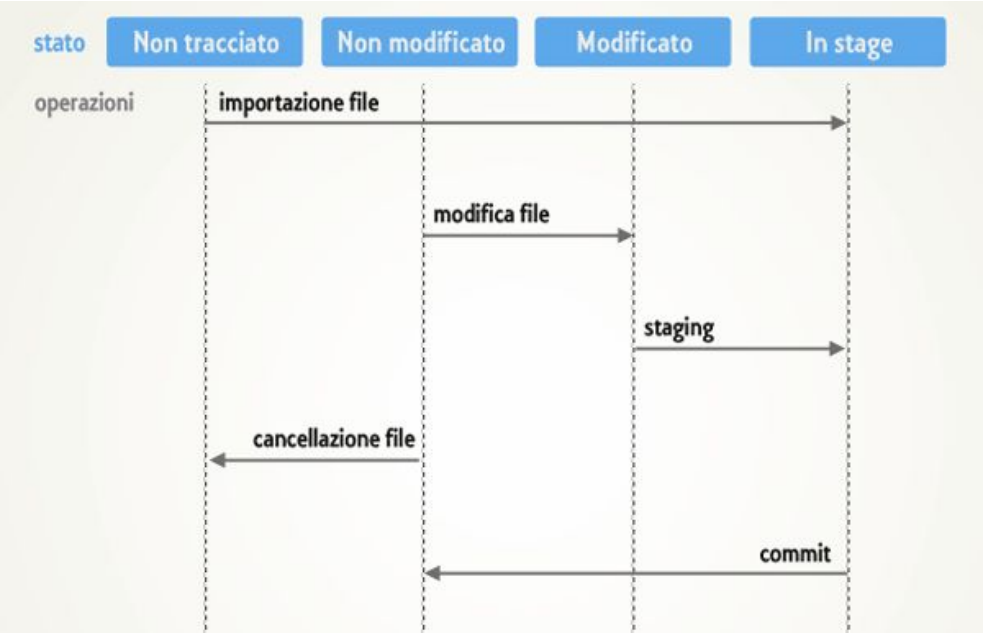
“The problem with Git jokes? Everybody has their own version”

Anonimo

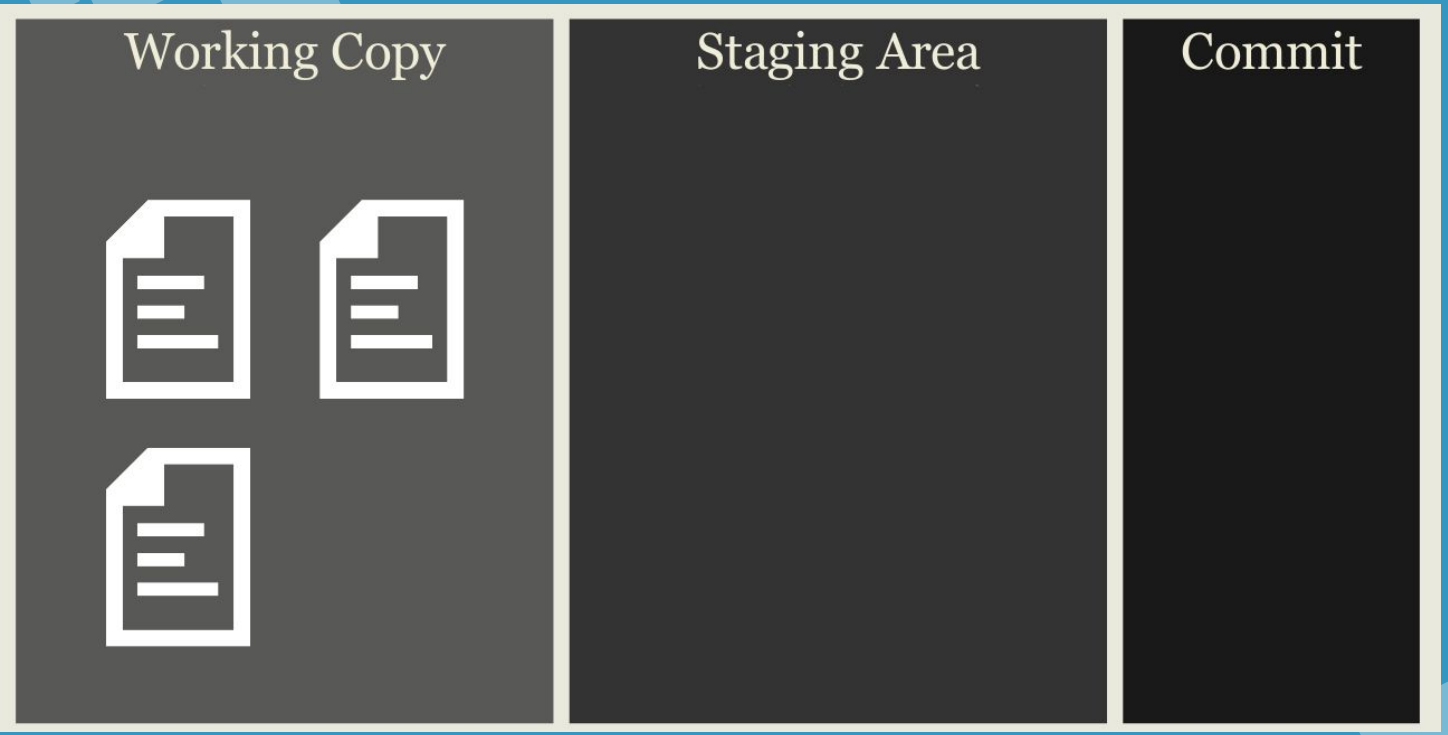
Glossario minimo

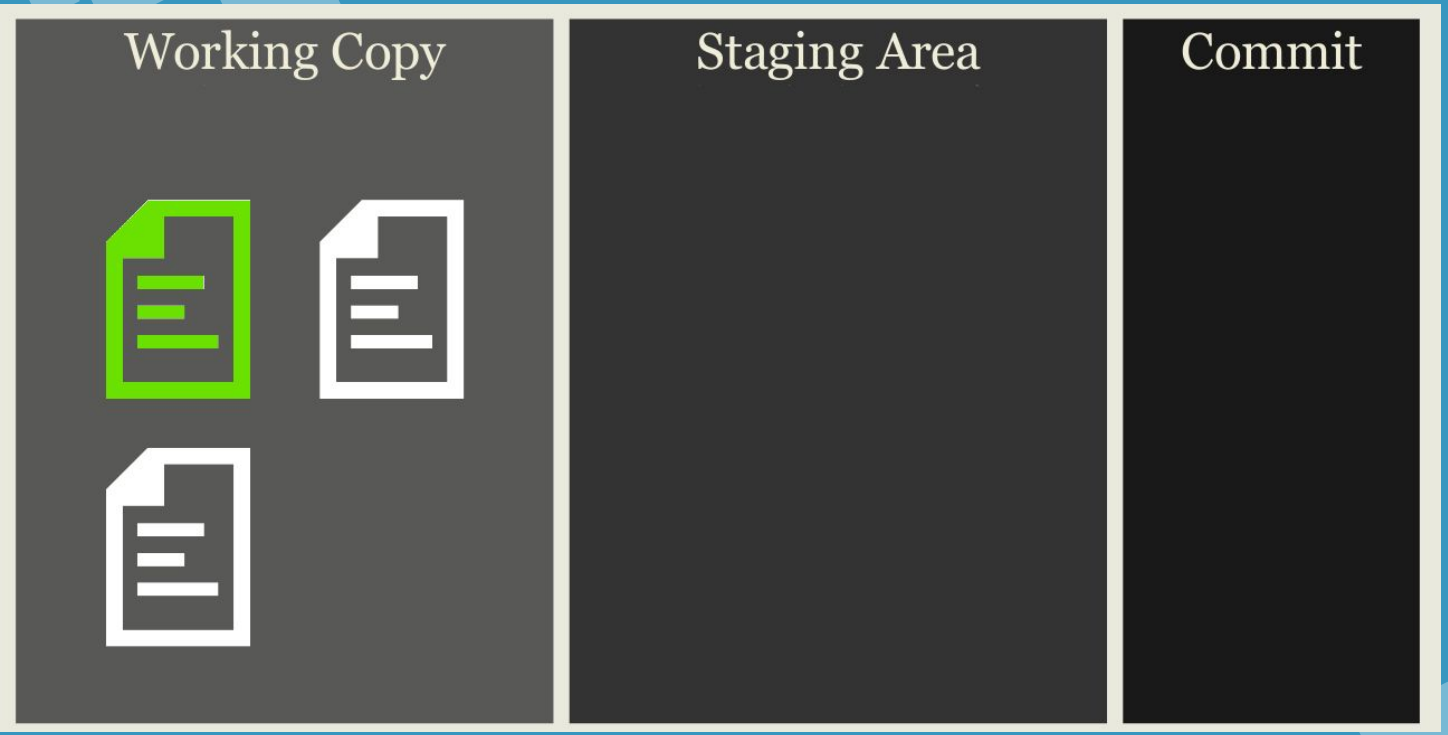
- **Repository** (deposito): un insieme di oggetti (file) e i riferimenti necessari - in pratica una “foto” del progetto. Può essere online o offline
- **Clone**: l'atto di duplicare localmente un repository
- **Commit** (lett. 'impegno'): Invio delle modifiche effettuate al repository
- **Checkout**: Ripristino di una particolare versione del progetto

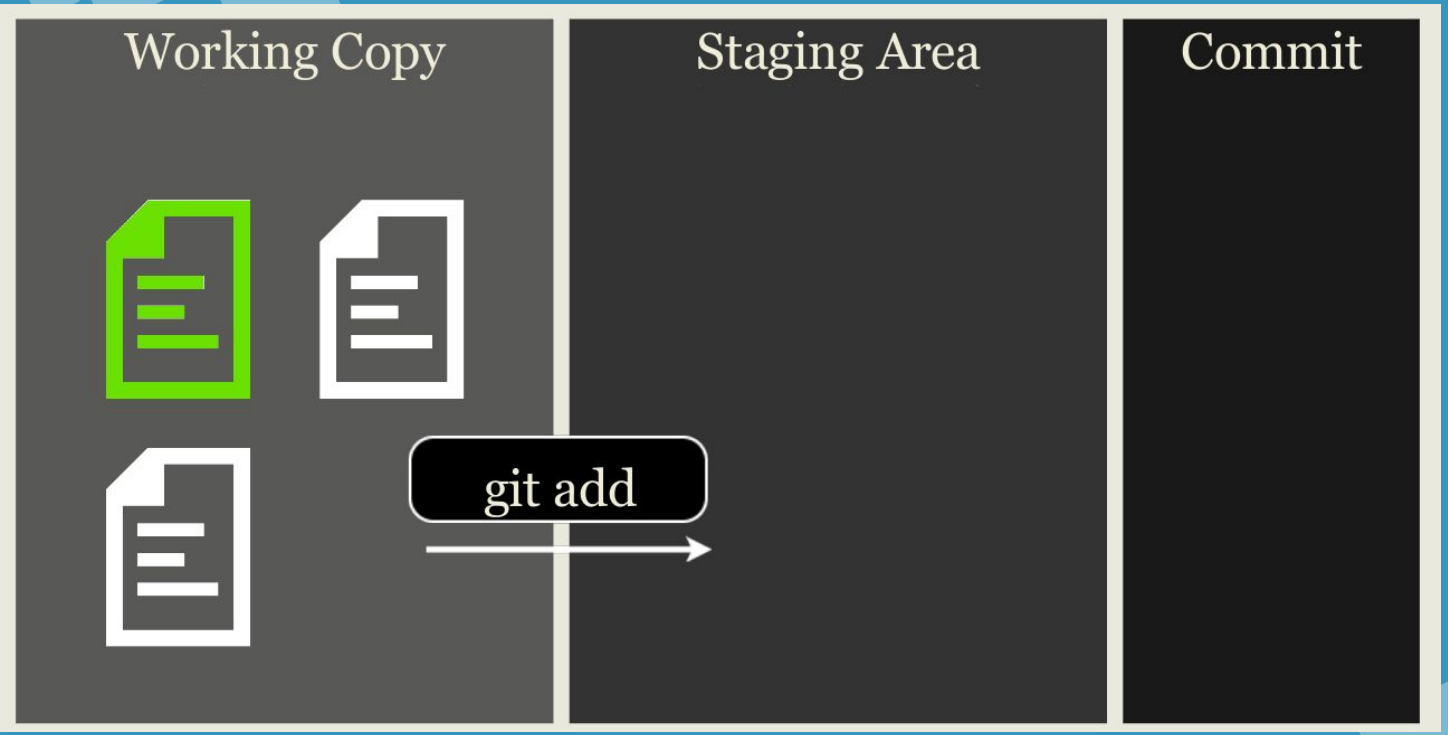
Stati dei file su git

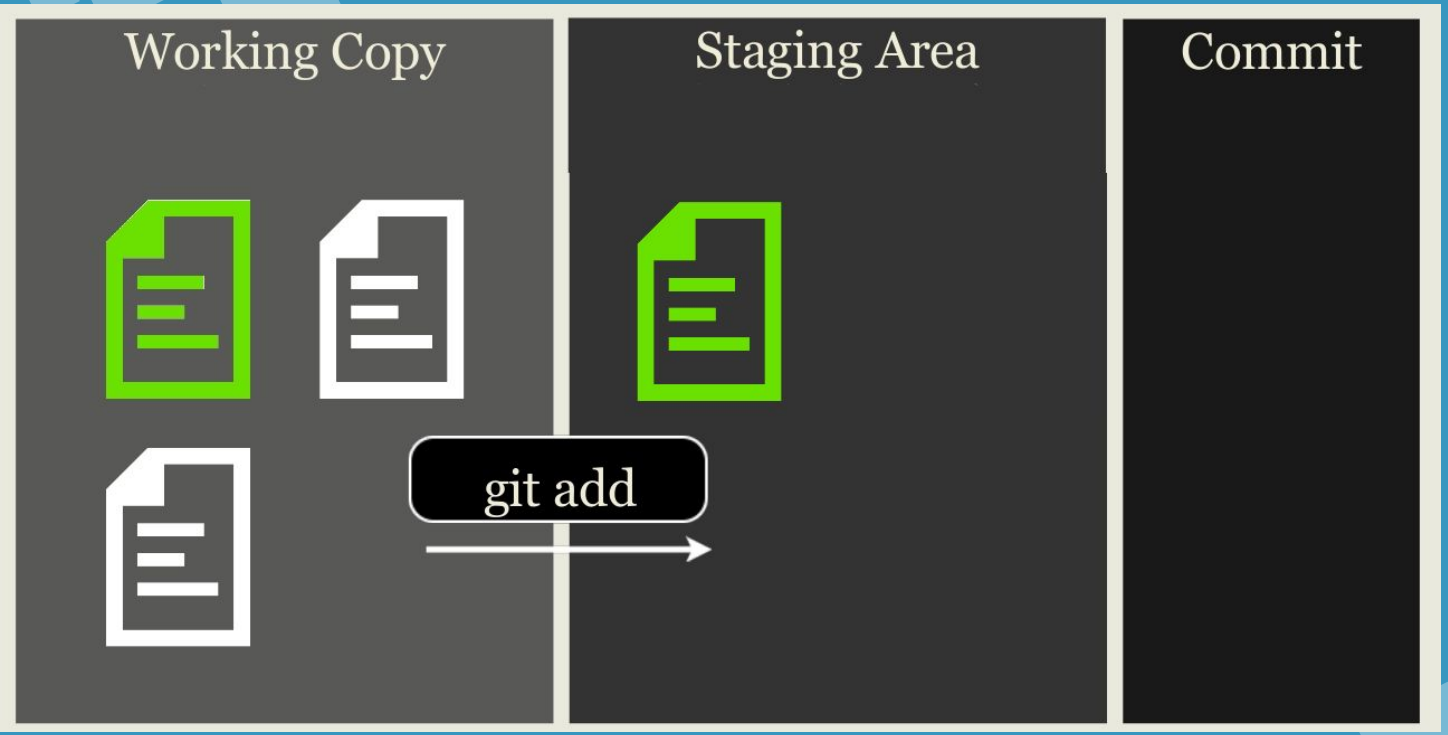


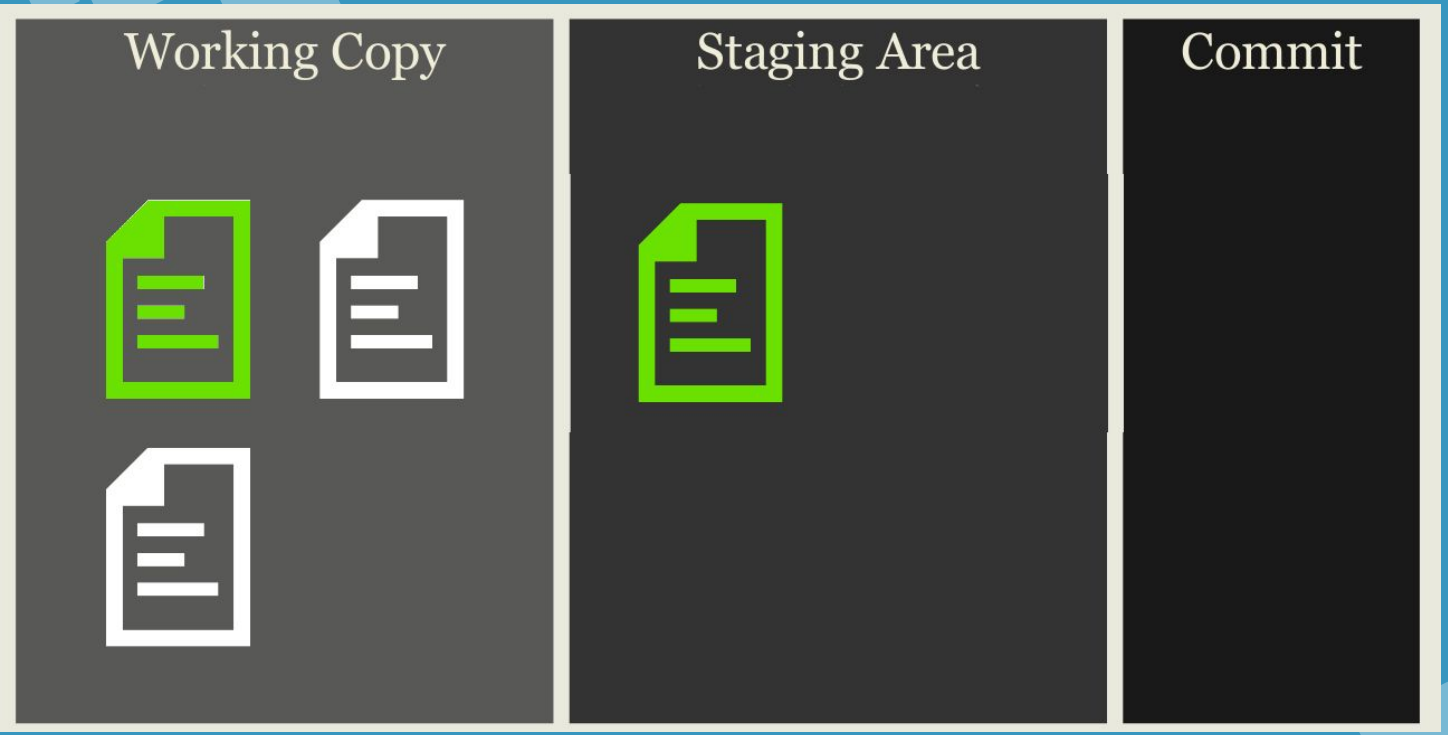
Sequenza di lavoro tipica

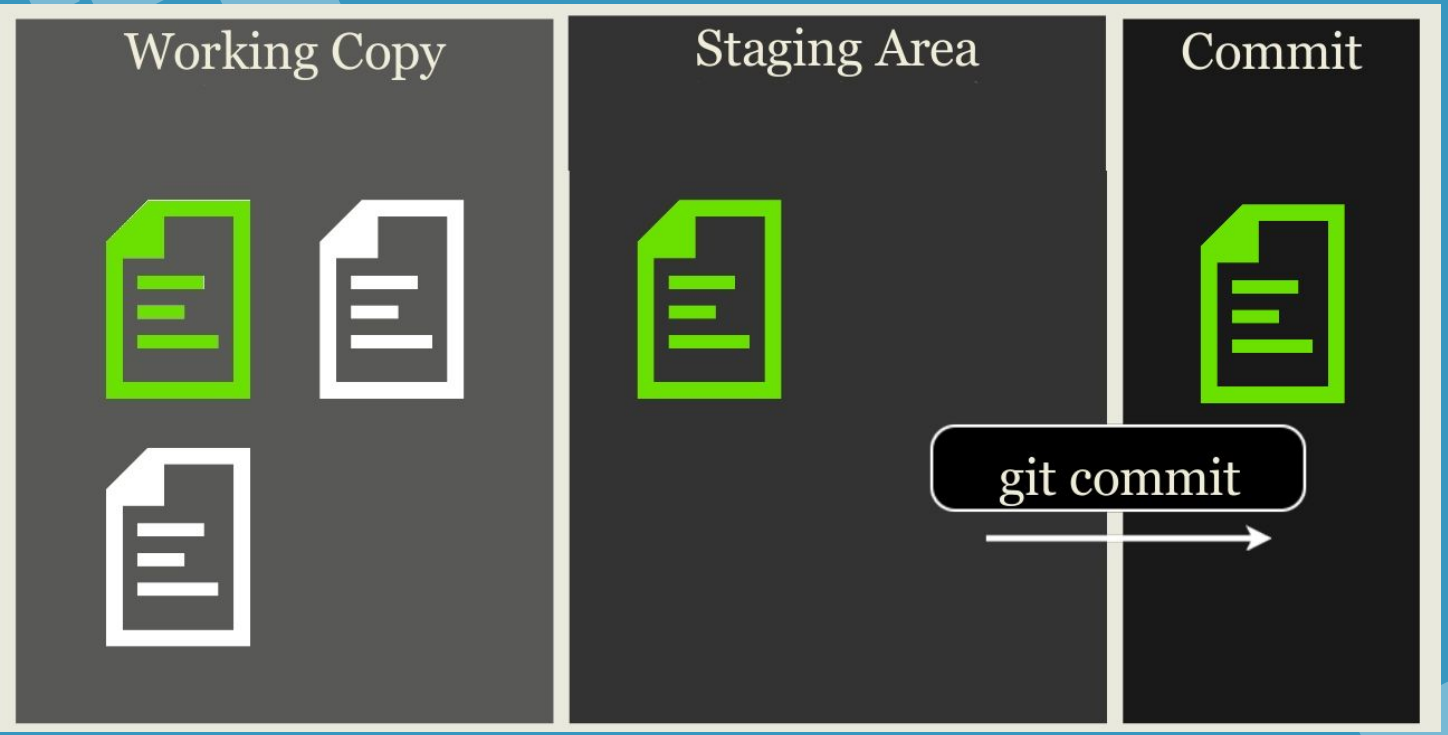

Working Copy
Staging Area
Commit


Working Copy
Staging Area
Commit

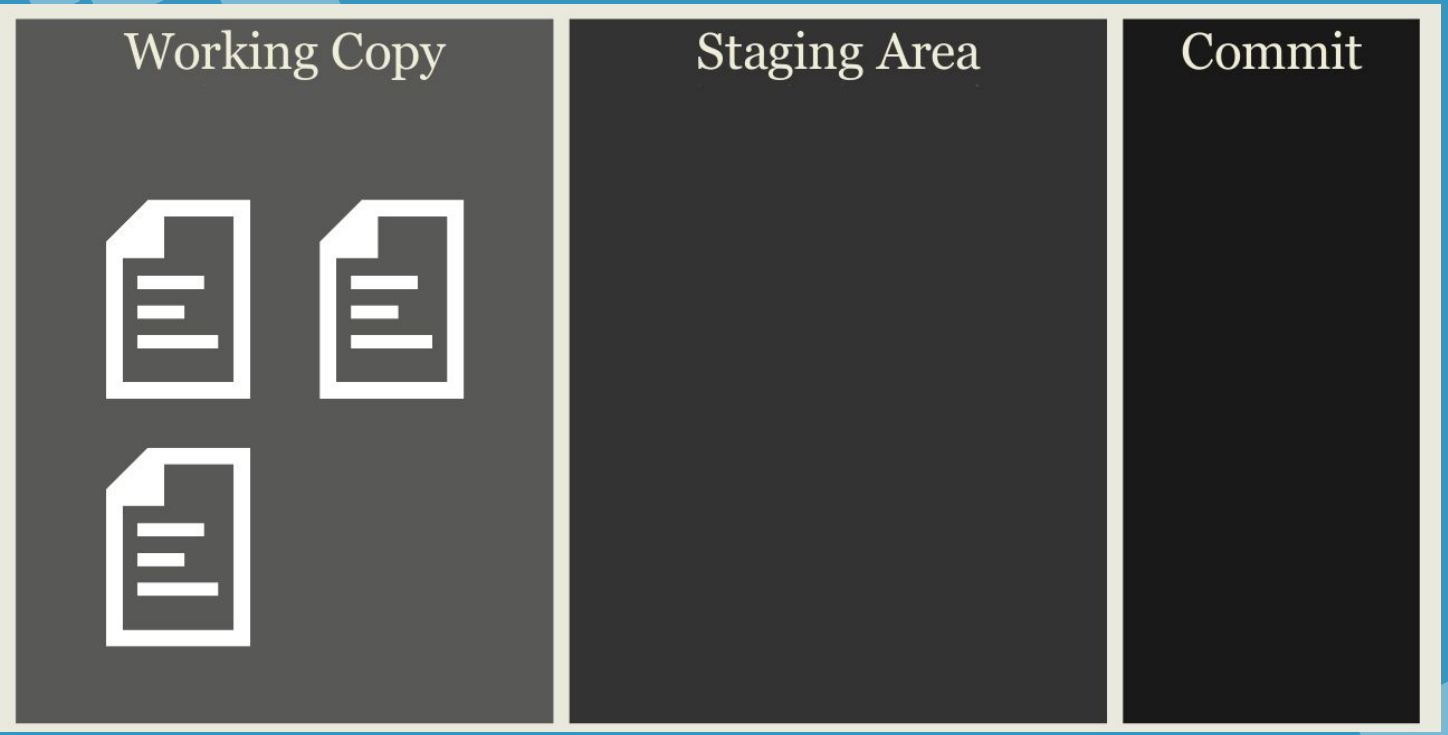

Working Copy
Staging Area
Commit
git add


Working Copy
Staging Area
Commit
git add


Working Copy
Staging Area
Commit


Working Copy
Staging Area
Commit
git commit

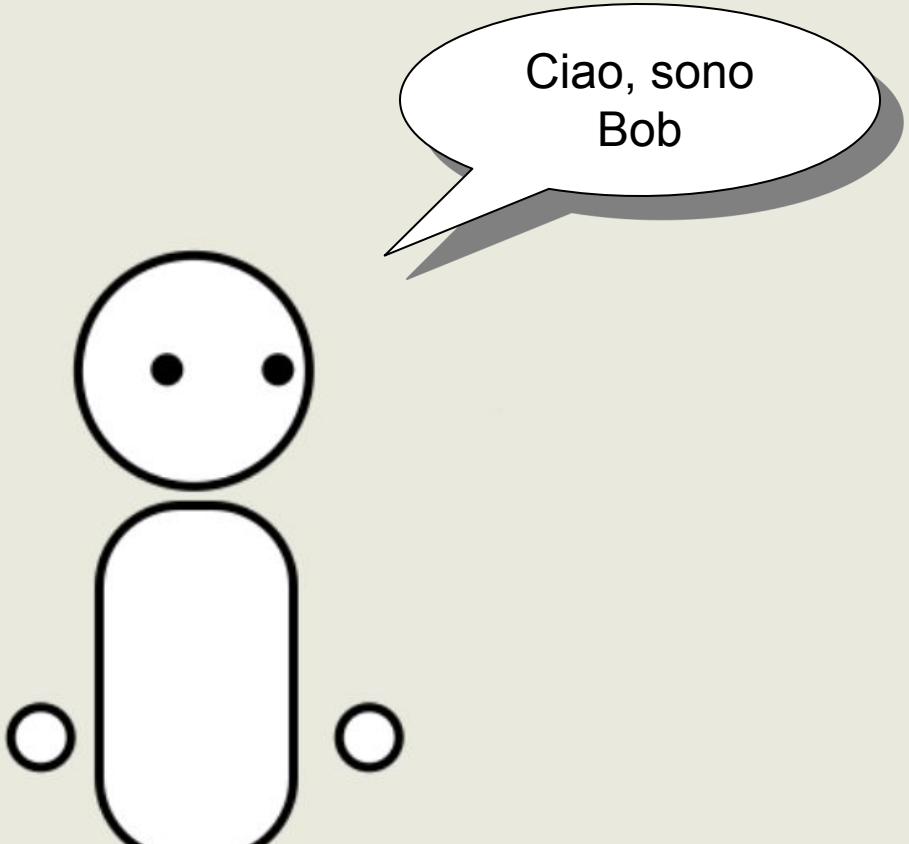
Working Copy
Staging Area
Commit


Working Copy
Staging Area
Commit

Chiesa di BOB

3.

Un esempio “fantasy”


Ciao, sono
Bob

Chiesa di BOB

La chiesa di Bob

Per imparare i rudimenti di Git, seguiremo i passi della inesistente "Chiesa di Bob", una religione bislacca creata proprio da Bob, un pazzoide con manie di grandezza.

Bob ha deciso intenzione di scrivere il suo personalissimo credo religioso in un libro, poco fantasiosamente chiamato “Libro di Bob”. Bob ha una discreta esperienza tecnica: presupporremo quindi che abbia installato un sistema con git perfettamente funzionante.

```
$ mkdir librosacro

$ cd librosacro

$ git init
```

```
Capitolo 1
NerdGenesi

All'inizio ci fu Bob, che
era l'origine di tutto.

'capitolo 1.txt' salvato
```

Status file: non tracciato

```
$ git status
# On branch master
#
# Initial commit
#
# Untracked files:
#
(use "git add <file>..." to
include in what will be committe
#
#
```

Status file: nella staging area

```
$ git add chapter1.txt
$ git status
# On branch master
#
# Initial commit
#
# Changes to be committed:
#
(use "git rm --cached <file>..."
to unstage)
#
```

Status file dopo il commit: non modificato

```
$git commit -m "prima versione"
[master (commit radice) 1f5a134]
 prima versione
 1 file changed, 6 insertions(+)
 create mode 100644 capitolo1.tx
```

Riassunto

Quello che abbiamo visto è un esempio tipico del ciclo di lavoro base di git. Ora proveremo ad applicarlo davvero con il nostro computer.

RIVEDIAMO QUALCHE CONCETTO

Repository

Insieme dei file del progetto con tutta la loro storia

Snapshot

La fotografia del progetto in un dato istante del tempo

Staging area

File pronti ad essere memorizzati.

Add

Aggiunta di file alla staging area

Commit

Conferma dei cambiamenti, il repo viene cambiato e la Staging Area svuotata

Checkout

Ripristino di una diversa versione.

GRAZIE!

Ma abbiamo appena cominciato….

CREDITS

Ringraziamenti a :

- Presentation template by SlidesCarnival
- Photographs by Unsplash
- Anil Gupta (www.guptaanil.com)
- Pete Nicholls (github.com/Aupajo)
- Armando Fox

Questo documento è distribuito con licenza CreativeCommon BY-SA 3.0